

Technical Cheat Sheet

Every term, model, method, dataset, and training decision in the project — defined and explained

1. Glossary of terms

Grouped by theme. Each entry: what it is, and how it is used in this project specifically.

1.1 Representations and spaces

Term	Definition and role in this project
Embedding / representation	A vector of numbers a model produces for an input, positioning it in a geometric space where distance ~ semantic dissimilarity. Everything in this project compares such vectors: 768-d hidden states (LLMs), 512-d MobileCLIP image vectors, 768-d SigLIP image/text vectors.
Activation / hidden state	The output of a specific layer for a specific input. LLMs produce one per token per layer; Experiment A stores one vector per passage per layer via mean pooling.
Mean pooling	Averaging token-level vectors over the sequence (masked to real tokens) to get one fixed-size vector per text. Lossy — discards word order — which makes Experiment A's similarity estimates conservative.
L2 normalization	Scaling a vector to unit length. After it, dot product = cosine similarity. All retrieval vectors in Experiment B are L2-normalized; the Core ML adapter normalizes its output.
Cosine similarity	Angle-based similarity in $[-1, 1]$, blind to vector length. The metric Qdrant search uses, hence the metric the adapter is judged by.
Anisotropy	Unequal spread of a space's variance across directions. MobileCLIP's and SigLIP's spaces differ in anisotropy — the reason rotation-only alignment (Procrustes) failed while full linear (ridge) succeeded.
Isomorphic (up to linear map)	Two spaces containing the same information in different coordinates, convertible by one matrix. The hypothesis asserts trained models' spaces are approximately this.
Permutation symmetry	Reordering a layer's neurons (with consistent rewiring) changes weights completely but not the function. The reason raw weights of independent models cannot be compared, averaged, or transplanted.

1.2 Measurement and statistics

Term	Definition and role in this project
Gram matrix	For n inputs, the $n \times n$ matrix of pairwise similarities as one model sees them — a coordinate-free fingerprint of the representation. CKA compares two models' Gram matrices.
CKA (Centered Kernel Alignment)	Similarity in $[0,1]$ between two representation sets, invariant to rotation, permutation, and uniform scaling — exactly the symmetries that leave a network's function unchanged. Used in A2 for every layer pair.
R^2 (coefficient of determination)	Fraction of the target's variance a prediction explains; 1 = perfect, 0 = as good as predicting the mean, negative = worse than the mean (seen in the starved run's random baseline). Used in A3 and B2 on held-out data.
Recall@K	Fraction of queries whose correct item appears in the top K results. The operational gate of Experiment B: adapter must keep $\geq 90\%$ of the ceiling's Recall.
Held-out / train-eval split	Fitting on one subset (75%) and measuring on untouched data (25%), so scores reflect generalization, not memorization. The same fixed split is reused across B2/B3/MLP for direct comparability.

Term	Definition and role in this project
Baseline (floor) / ceiling	The score with no learning (random weights; raw cross-space retrieval) vs the best achievable (native SigLIP). Every result in the project is read as its position between the two.
Sample-to-dimension ratio	Training rows per input dimension of a fitted map. At 1.6 (run 1) ridge is starved and systematically understates alignment; at ~10 (run 2) estimates stabilize. The project's key methodological lesson.
Overfitting / early stopping	Fitting noise beyond signal; prevented in the MLP comparison by keeping the checkpoint with best held-out cosine (reached at epoch 30 of 300).

1.3 Maps and training

Term	Definition and role in this project
Reparameterization (structural)	Folding a multi-branch training-time block (parallel convs + BatchNorm + identity) into a single equivalent convolution at inference. Used throughout MobileCLIP-S1; the reason its deployed graph is simpler and faster than its printed module tree, and why the ANE compiles it efficiently.
Identity normalization	A preprocessing Normalize step with mean 0 and std 1, i.e. no change. MobileCLIP-S1 uses it (raw [0,1] pixels); replicating the pipeline incorrectly on device silently invalidates the adapter's calibration.
Adapter (linear map)	Here: a single 512×768 matrix W translating MobileCLIP image vectors into SigLIP space: $y = L2norm(W \cdot x)$. 1.6 MB fp32 / 0.79 MB fp16. The project's deliverable.
Ridge regression	Linear least squares with an L2 penalty $\alpha \cdot I$ stabilizing the inverse: $W = (X^T X + \alpha I)^{-1} X^T Y$. Closed-form — solved exactly in one line, no gradient descent. Fits both the stitching maps (A3) and the adapter (B2) in seconds.
Closed-form solution	An answer computed by formula rather than iterative optimization. Why 'training' the adapter costs seconds and is exactly reproducible.
Orthogonal Procrustes	The best pure rotation aligning two point sets, from the SVD of $X^T Y$. Tests the stronger 'rigidly same shape' claim; succeeded implicitly in A (spaces near-rotational), failed in B ($R^2 = 0.038$) due to anisotropy differences.
MLP (multi-layer perceptron)	A small neural network (here 512->1024->768 with GELU) used as a controlled upgrade attempt. Its +0.003 non-improvement proved the linear map optimal.
GELU	A smooth activation function standard in transformers; the nonlinearity inside the comparison MLP.
Cosine loss	$1 - \cos(\text{prediction}, \text{target})$; trains the MLP directly on the quantity retrieval cares about, rather than MSE.
Model stitching	Connecting layer i of model A to layer j of model B through a small trained map and measuring the penalty. A3 is stitching with closed-form ridge and R^2 as the penalty measure.
Fine-tuning vs from-scratch	Fine-tuning updates an existing checkpoint (coordinates inherited); from-scratch starts from random init (coordinates independent). Both Experiment A models are from-scratch — the property that makes their alignment evidence of convergence.

1.4 Systems and deployment

Term	Definition and role in this project
Qdrant / vector database	A database that stores embeddings and answers nearest-neighbor queries. The target architecture: one collection, 768-d, cosine metric, written by both tiers.
HNSW	Hierarchical Navigable Small World — the graph index structure making approximate nearest-neighbor search fast; one shared graph instead of two is part of the efficiency payoff.

Term	Definition and role in this project
Core ML / .mlpackage	Apple's on-device inference format and runtime. B4 exports the adapter as an .mlpackage (MatMul + L2-normalize) via coremltools' PyTorch->MIL conversion.
Neural Engine (ANE)	The iPhone's NPU; runs fp16 Core ML models at very low energy. Executes both MobileCLIP and the adapter.
fp16 / fp32	16- vs 32-bit floating point. The adapter ships fp16 (0.79 MB, precision loss < 1e-3 — negligible for cosine ranking).
DATA_DIR convention	All pipeline stages read/write artifacts (activations.npz, pairs.npz, adapter.npz) under one configurable directory — on Colab, a mounted Google Drive path — making every stage stateless and VM-restart-proof.
Streaming / parquet datasets	Hub-hosted columnar data readable without custom loader scripts. The project migrated to parquet (wikitext) and direct official downloads (COCO) after HF deprecated script-based datasets mid-project.

2. Models: full profiles

Model	Architecture & size	Training data / objective	Role here
GPT-2 (124M)	Decoder-only transformer, 12 blocks, d=768, learned positional embeddings, GPT-2 BPE tokenizer (50,257 vocab). OpenAI, 2019.	WebText (~40 GB of Reddit-linked pages); next-token prediction.	Experiment A, model A. Frozen; only hidden states read.
Pythia-160M	GPT-NeoX decoder-only transformer, 12 blocks, d=768, rotary positional embeddings, NeoX tokenizer. EleutherAI, 2023.	The Pile (~825 GB, 22 diverse sources); next-token prediction. Trained from scratch — no shared ancestry with GPT-2.	Experiment A, model B. Frozen. The LM-head weight (embed_out) is discarded on load — the benign UNEXPECTED warning.
Random-weights Pythia	Identical architecture to Pythia-160M, weights freshly sampled from the init distribution; zero gradient steps.	None — that is the point.	The control: measures similarity obtainable from architecture + shared inputs alone. Trained-minus-random = effect of learning.
MobileCLIP-S1	Apple's efficient image-text model; hybrid conv-transformer image tower (FastViT/MC1), 21.5 M vision parameters verified by introspection; channel pyramid 64-128-256-512 with 4/12/20/4 blocks per stage; attention only in the final stage; embedding d=512. Designed for the ANE; loaded via open_clip (pretrained 'datacompdr').	Contrastive image-text training on DataCompDR (reinforced web image-text pairs).	The phone-tier encoder. Frozen; source space of the adapter.
SigLIP 2 (base, patch16-224)	Google's vision-language model; ViT-Base image tower (16px patches, 224px input), paired text tower; both embeddings d=768; ~1.5 GB checkpoint.	Sigmoid-loss contrastive image-text training (per-pair logistic objective instead of softmax) — the reason its absolute cosines run low (~0.15 diagonal) while ranking stays sharp.	The server-tier encoder and the target space. Frozen; its text tower produces all retrieval queries and is unaware the adapter exists.

Table: the five models. 'Frozen' = no weights updated anywhere in the project.

Encoders, precisely: an *encoder* is a model (or tower) mapping raw input to an embedding. The project uses four: GPT-2 and Pythia as text encoders (reading all 13 hidden-state levels, not just the last), MobileCLIP's image tower, and SigLIP's image + text towers. Decoding/generation is never used anywhere.

2.1 Inside MobileCLIP-S1 (verified by introspection)

Two structural facts matter for this project. **(a) Reparameterizable convolutions.** Nearly every block is a MobileOne/RepMixer unit holding parallel branches during training (3x3 conv + 1x1 conv + identity, each with BatchNorm) that fold algebraically into a single convolution at inference - training-time capacity at deployment-time cost. BatchNorm rather than LayerNorm throughout is what makes the fold possible. Attention appears only in the last stage (4 blocks, QKV 512 -> 1536) where the grid is already 8x8, so global mixing is nearly free. **(b) A one-layer bottleneck.** The trunk carries 1024 channels until a single final projection Linear(1024 -> 512) emits the embedding - so the capacity gap that limits R@1 is concentrated in one layer, and a wider-output sibling (MobileCLIP-B, MobileCLIP2) is the predicted remedy.

2.2 Preprocessing - a silent-failure trap

MobileCLIP-S1's transform is: bilinear Resize(256), CenterCrop(256x256), tensor conversion, and **Normalize(mean=(0,0,0), std=(1,1,1)) - an identity operation**. The model consumes raw [0,1] pixels, unlike standard CLIP models that subtract dataset means. Since the adapter matrix W is fitted against the embedding distribution this exact pipeline produces, replicating it inexactly on iOS (for example by copying conventional CLIP normalization constants) shifts every vector and degrades retrieval with no error raised. The iOS client must reproduce `resize-256 / center-crop / [0,1]` scaling and nothing else.

3. Data inputs: what, why, how

Dataset	Contents & size used	Why chosen	How used
WikiText-103 (Salesforce/wikitext, train split)	Long-form Wikipedia articles; 10,000 passages > 100 chars (run 2; run 1 used 1,646 validation passages).	Diverse, clean English prose; parquet-hosted (survives the datasets-scripts deprecation). Content is out-of-domain for neither model, favoring neither.	Identical passages fed to GPT-2, Pythia, and the random control; truncated at 64 tokens; batch 32; mean-pooled per layer -> activations.npz (3 × [13, 10000, 768]).
COCO val2017 (official cocodataset.org zips)	5,000 photos with 5 human captions each; 4,000 image-caption pairs used (first caption per image); ~1 GB images + 240 MB annotations.	Natural photos + human English captions — the adapter's actual production domain; standard retrieval benchmark, so results are sanity-checkable against published numbers; plain zips immune to loader-script deprecations.	Each image encoded by both image towers; each caption by SigLIP's text tower -> pairs.npz (mob_img 4000×512, sig_img 4000×768, sig_txt 4000×768), split 3000 train / 1000 eval.

Input processing details: LLM inputs are tokenized by each model's own tokenizer (deliberately different — part of the independence claim); attention-masked mean pooling excludes padding. Images are preprocessed by each encoder's own pipeline (resize/normalize per its training recipe); captions are tokenized by SigLIP's tokenizer, padded to 64. All embeddings are L2-normalized at extraction so every later stage works in cosine geometry.

4. Methods, step by step

4.1 CKA computation (A2)

For layer i of model A (matrix X , 10000×768) and layer j of model B (Y): center columns; $CKA = ||Y^T X||_F^2 / (||X^T X||_F \cdot ||Y^T Y||_F)$ — algebraically the cosine between the two centered Gram matrices. Computed for all 13×13 pairs, twice (trained-vs-trained, trained-vs-random). Read as: high diagonal = same content developed in the same order through depth.

4.2 Linear stitching (A3)

Per matched layer pair: 75/25 split; standardize inputs; solve ridge $W = (X^T X + aI)^{-1} X^T Y$ with $a=1.0$ on train; report variance-weighted R^2 on eval. Layer matching is one-to-one here (both models have 13×768 states); the code supports proportional matching for unequal depths. The run-1 -> run-2 comparison changed only N (1,646 -> 10,000) and the source split.

4.3 Adapter fitting (B2)

Ridge on raw (unstandardized, already L2-normalized) pairs with $a=0.01$: $W = (X^T X + aI)^{-1} X^T Y$, $X = \text{mob_img}[\text{train}]$ (3000×512), $Y = \text{sig_img}[\text{train}]$. Procrustes variant: zero-pad X to 768, $\text{SVD}(X^T Y) = U S V^T$, $R = U V^T$. Both evaluated on the same 1,000 held-out images by R^2 and mean cosine-to-target.

4.4 Retrieval evaluation (B3)

Queries: SigLIP text embeddings of the 1,000 eval captions (exactly the production query path). Galleries: (A) native sig_img — ceiling; (B) $L2\text{norm}(\text{mob_img} @ W)$ — the system; (C) raw mob_img zero-padded — chance floor. Similarity = dot product (all unit-norm); ground truth is the diagonal (caption k matched to image k); $\text{Recall}@\{1,5,10\}$ = fraction of captions ranking their own image in top K . Gate: $B \geq 90\%$ of A per K .

4.5 MLP comparison (B2-bis)

`nn.Sequential(Linear 512->1024, GELU, Linear 1024->768)`; AdamW lr $1e-3$, weight decay $1e-4$; cosine LR schedule; batch 256; 300 epochs; loss = $1 - \text{cosine}(\text{normalized prediction}, \text{target})$; early stopping keeps the best held-out-cosine checkpoint (epoch ~30). Same split as B2. Result: +0.003 cosine, retrieval unchanged -> linearity established.

4.6 Core ML export (B4)

Wrap W in a bias-free torch Linear plus `F.normalize`; `torch.jit.trace` on a dummy (1×512); `coremltools.convert` with FLOAT16 precision, iOS16 target; save Adapter.mlpackage (0.79 MB). Conversion path: PyTorch graph -> MIL ops -> mlprogram. Parity check on macOS: max |CoreML - numpy| expected < $1e-3$ (fp16).

5. What was trained — and what deliberately was not

Component	Trained?	How / why not
GPT-2, Pythia, MobileCLIP, SigLIP 2	No — frozen throughout	The entire point: interoperability without touching the models.
Random-weights Pythia	No — never trained by design	Control condition isolating the effect of learning.
13 stitching matrices (A3)	Closed-form fit (seconds)	Ridge regression; 'trained' only in the loosest sense — no epochs, no gradients.
Adapter W (B2)	Closed-form fit (seconds)	Ridge on 3,000 pairs; refittable in seconds whenever either encoder is upgraded.
Procrustes R (B2)	Closed-form fit (seconds)	SVD; tests the rotation-only hypothesis.

Component	Trained?	How / why not
Comparison MLP (B2-bis)	Yes — 300 epochs, ~2 min GPU	The only gradient training in the project, run deliberately as a control to prove it was unnecessary.

Total compute for the full study — extraction, two A-runs, all fits, MLP, export — is under one GPU-hour on a free Colab T4.

Addendum — Experimental design vocabulary

Terms used when discussing why Experiment A was run the way it was (see report Appendix C).

Term	Meaning
Train / validation / test split	Partitions of a dataset defined by how much information reaches the model: train (weights fitted on it), validation (guides the experimenter's choices), test (used once, for the honest final number). The roles presuppose a model being trained - they carry no meaning for frozen models used only to read activations.
Probe stimuli	Inputs fed to a frozen model purely to elicit internal activations for measurement, analogous to a stimulus set in a perception experiment. WikiText plays this role in Experiment A: it trains nothing.
Held-out evaluation	Measuring a fitted map on rows it never saw. A 768x768 matrix with 590K free parameters can memorize a small sample exactly; in-sample R2 would then be meaningless. Every R2 in this project is held-out.
Samples per dimension	Training rows divided by input dimensionality - the practical measure of statistical power for a linear fit. Below roughly 5, ridge shrinkage dominates and estimates become systematically pessimistic. Run 1: 1.6. Run 2: 9.8.
Shrinkage bias (starved regression)	When rows are scarce relative to parameters, the ridge penalty forces coefficients toward zero to stay stable. The fit generalizes poorly in one direction: it understates the true relationship. Negative held-out R2 on a control is its signature - the fit generalizes worse than predicting the mean.
Pre-registration	Fixing success criteria before seeing results, so that a verdict cannot be chosen after the fact. Used for both experiments; Appendix C also records the one sub-threshold that was missed.
Diagnosed confound	A specific, checkable reason a measurement may be wrong, identified before rerunning and paired with a prediction that distinguishes it from the alternative. What separates a correction from fitting the result.

Rule of thumb. Before accepting any negative result from a fitted map, check that the measurement had the power to show a positive one: rows per parameter first, then alignment of the two data matrices, then the pooling or preprocessing that produced them.